

UNIT IV

Efficient Test Suite Management: growing nature of test suite, Minimizing the test suite and its benefits, test suite prioritization, Types of test case prioritization, prioritization techniques, measuring the effectiveness of a prioritized test suite

Software Quality Management: Software Quality metrics, SQA models.

Efficient Test Suite Management

Test Case Design

- Test design is the act of creating and writing test suites for testing software.
- Test analysis and identifying test conditions gives us a generic idea for testing which covers a large range of possibilities.
- But when we make a test case we need to be very specific. It need the exact and detailed specific input.
- But just having some values to input to the system is not a test, if we don't know what the system is supposed to do with the inputs, we will not be able to tell that whether our test has passed or failed.
- Test cases can be documented as described in the IEEE 829 Standard for Test Documentation.
- It consists of different documents covering: Test plan, Test Design Specification, Test Case Specification, Test Procedure Specification, Test Item Transmittal Report, Test Log, Test Incidental Report, Test Summary Report.

1. Growing Nature of Test Suite

- There may be unnecessary test cases in the test suite including both obsolete (out of date) and redundant test cases.
- For example a change in a program causes a test case to become obsolete.
- A test case is redundant if other test cases in test suite provide the same coverage of the program.
- Thus due to obsolete and redundant test cases, the size of a test suite continues to grow unnecessarily.
- Test engineers measures the extent to which a criterion is satisfied in terms of **Coverage**. a test set achieves 100% coverage if it completely satisfies the criterion.
- Coverage is measured in terms of the requirements that are imposed.
- Partial Coverage is defined as the percentage of requirements that are satisfied.
- Coverage Criteria is used as a stopping point to decide when a program is sufficiently tested.

- For example, to achieve statement coverage for a program, one would add additional test cases to the test suite until each statement in that program is executed by at least one test case.

2.Minimizing the test suite and its benefits

- A test suite can sometimes grow to an extent that it is nearly impossible to execute.
- In this case, it becomes necessary to minimize the test cases such that they are executed for maximum coverage. The reasons why minimization is important are:
 - Release date of the product is near.
 - Limited staff to execute all the test cases.
 - Limited test equipments or unavailability of testing tools

Minimizing the test suite has the following Benefits:

- Redundant test cases will be eliminated.
- Lower costs by reducing a test suite to a minimal subset.
- A reduction in the size of test suite decreases both the overhead of maintaining the test suite and the number of test cases that must be rerun after changes are made to the software, thereby reducing the cost of regression testing.

3.Test Suite Prioritization

- The purpose of prioritization is to reduce the number of test cases based on some criteria, while aiming to select the most appropriate tests. The different priorities are:

Priority1: The test cases that must be executed otherwise there may be worse consequences for the release of the product.

Priority 2: The test cases may be executed, if time permits.

Priority 3: The test case is not important prior to the current release. It may be tested shortly after the release of the current version of the software.

Priority 4: The test case is never important as its impact is nearly negligible.

4.Types of test case prioritization

General Test Case Prioritization:

- In this prioritization, we prioritize the test cases that will be useful over a succession of subsequent modified versions of P (Program or Product), without any knowledge of the modified versions.

Version-Specific Test Case Prioritization:

- Here, we prioritize the test cases such that they will be useful on a specific version P' of P.

5.Prioritization Techniques

- Prioritization techniques schedule the execution of test cases in an order that attempts to increase their effectiveness at meeting some performance goal.
- Therefore, given any prioritization goal, various prioritization techniques may be applied to a test suite with the aim of meeting that goal.
- For example, in an attempt to increase the rate of fault-detection of test suites, we might prioritize test cases in terms of they execute modules that, measured historically, tend to fail.
- The intent behind the choice of a prioritization technique is to increase the prioritized test suite can better meet the goal than an *ad hoc* or random ordering of test cases.
- Prioritization can be done at two levels, they are:

Prioritization for regression test suite

- This category prioritizes the test suite of regression testing is performed whenever there is a change in the software; we need to identify the test cases corresponding to modified and affected modules.

Prioritization for system test suite

- This category prioritizes the test suite of system testing. Here, the consideration is not the change in the modules.
- The test cases of system testing are prioritized based on several criteria: risk analysis, user feedback, fault-detection rate, etc.

1. COVERAGE-BASED TEST CASE PRIORITIZATION

2. RISK-BASED PRIORITIZATION

3.BASED ON OPERATIONAL PROFILES

4.PRIORITIZATION USING RELEVANT SLICES

5. PRIORITIZATION BASED ON REQUIREMENTS

1.Coverage Based Test Prioritization

- This type of prioritization is based on the coverage of codes, such as statement coverage, branch coverage, etc. and the fault exposing capability of the test cases.
- For example, count the number of statements covered by the test cases. The test case that covers the highest number of statements will be executed first.
- Some of the techniques are:
 - 1.Total Statement Coverage Prioritization
 - 2.Additional Statement Coverage Prioritization
 - 3.Total Branch Coverage Prioritization
 - 4.Additional Branch Coverage Prioritization

5.Total Fault-Exposing-Potential (FEP) Prioritization

1.Total Statement based Prioritization:

- This prioritization orders the test cases based on the total number of statements covered.
- It counts the number of statements covered by test cases and orders them in descending order.
- For example, if T1 covers 5 statements, T2 covers 7 statements and T3 covers 14 statements, then the order of prioritization is T3, T2, T1.

2.Additional Statement Coverage Prioritization:

- Additional statement coverage prioritization iteratively selects a test case *T1*, that yields the greatest statement coverage, then selects a test case which covers a statement uncovered by *T1*.
- Repeat this process until all statements covered by at least one test case have been covered.
- For example, if we consider the below Table according to total statement coverage criteria (a principle) the order is 2,1,3.
- But additional statement coverage selects test case 2 first and next, it selects test case 3, as it covers statement 4 which has not been covered by test case 2.
- Thus, the order according to additional coverage criteria is 2,3,1.

Statement	Statement Coverage		
	Test case 1	Test case 2	Test case 3
1	X	X	X
2	X	X	X
3		X	X
4			X
5			
6		X	
7	X	X	
8	X	X	
9	X	X	

Statement coverage

3.Total Branch coverage Prioritization:

- In this prioritization, the criterion to order is to consider condition branches in a program instead of statements.
- Thus it is the coverage of each possible outcome of a condition in a predicate.
- The test case which will cover maximum branch outcomes will be ordered first.

- For example, in the following diagram the order will be test cases 1,2,3.

Branch Statements	Branch Coverage		
	Test case 1	Test case 2	Test case 3
Entry to while	X	X	X
2-true	X	X	X
2-false	X		
3-true		X	
3-false	X		

Branch coverage

4. Additional Branch Coverage Prioritization

- The idea is the same as in additional statement coverage of first selecting the test case with the maximum coverage of branch outcomes and then, selecting the test case which covers the branch outcome not covered by the previous one.

5. Total Fault-Exposing-Potential (FEP) Prioritization:

- Statement and branch coverage prioritization ignore a fact about test cases and faults.
 - Some bugs/faults are more easily uncovered than other faults.
 - Some test cases have the proficiency (expertise) to uncover particular bugs as compared paped.
- Thus the ability of a test case to uncover a fault is called the **fault exposing potential**.
- To obtain an approximation of a test case, an approach was adopted using manual analysis. The approach is: Given a program P and a test suite T,
 - First, create a set of mutants (resulting from) $N = \{n_1, n_2, \dots, n_m\}$ for P, noting which statement s_j in P contains each mutant.
 - Next, for each test case $t_i \in T$, execute each mutant version n_k of P on t_i , noting whether t_i kills that mutant.
 - Calculate the $FEP(s, t)$ as the ratio : Mutants of s_j killed / Total number of mutants of s_j .
- To perform total FEP prioritization, given these $FEP(s, t)$ values, calculate an **award value** for each test case $t_i \in T$, by summing the $FEP(s_j, t_i)$ values for all statements s_j in P.
- Given these award values, we prioritize test cases by sorting them in order of descending award value.
- Total FEP prioritization outputs the test case order as (2, 3, 1).

Statement	FEP(s,t) values		
	Test case 1	Test case 2	Test case 3
1	0.5	0.5	0.3
2	0.4	0.5	0.4
3		0.01	0.4
4		1.3	
5			
6	0.3		
7	0.6		0.1
8		0.8	0.2
9			0.6
Award values	1.8	3.11	2.0

2.Risk Based Prioritization

- Risk-based prioritization is a process that prioritizes modules for testing. using risk analysis to highlight potential problem areas, whose failures have adverse (harmful) consequences.
- The testers use this risk analysis to select the most crucial tests.
- Thus, risk-based technique is to prioritize the test cases based on some potential problems which may occur during the project.
- The two components of risk based technique are:

Probability of occurrence / fault likelihood: It indicates the probability of occurrence of a problem.

Severity of impact / failure impact: If the problem has occurred, how much impact does it have on the software.

- A risk analysis table consists of the following columns: Problem ID, Potential Problem, Uncertainty factor, Severity of impact, Risk Exposure.
- For example, the problems in the given table can be prioritized in the order of P5, P4, P2, P3 and P1 based on the risk exposure.

Problem ID	Potential Problem	Uncertainty Factor	Risk Impact	Risk Exposure
P_1	Specification ambiguity	2	3	6
P_2	Interface problems	5	6	30
P_3	File corruption	6	4	24
P_4	Databases not synchronized	8	7	56
P_5	Unavailability of modules for integration	9	10	90

Risk analysis table

3. Prioritization Based On Operational Profiles

- The prioritization of test cases is done based on the operation profiles of the important functions which are of use to the customer.
- An operational profile is a set of tasks performed by the system and their probabilities of occurrence.
- After estimating the operational profiles, testers decide the total number of test cases, keeping in view the costs and resource constraints.

4. Prioritization using Relevant Slices

- During regression testing, the modified program is executed on all existing regression test cases to check that it still works the same way as the original program, except where a change is expected.
- But re-running the test suite for every change in the software makes regression testing a time-consuming process.
- If we can find the portion of the software which has been affected with the change in software, then we can prioritize the test cases based on this information. This is called the **slicing technique**

Execution Slice:

- The set of statements executed under a test case is called **Execution Slice** of the program.
- For example, the following is the example program for execution slice

```
Begin
S1: read (basic, empid);
S2: gross = 0;
S3: if (basic > 5000 || empid > 0)
    {
S4:     da = (basic*30)/100;
S5:     gross = basic + da;
    }
S6: else
    {
S7:     da = (basic*15)/100;
S8:     gross = basic + da;
    }
S9: print (gross, empid);
End
```

- The test cases for the given program.

Test cases

Test Case	Basic	Empid	Gross	Empid
T1	8000	100	10400	100
T2	2000	20	2300	20
T3	10000	0	13000	0

- T1 and T2 produce correct results. T3 produces an incorrect result.
- Syntactically, it is correct, but an employee with the empid '0' will not get any salary, even if his basic salary is read as input. So it has to be modified.
- Suppose S 3 is modified as [if (basic>5000 && empid>0)]. It means for T 1, T 2, and T 3, the program would be rerun whether the change in S3 has introduced new bugs or not.
- But if there is a change in S 7 statement [da = (basic*25)/100; instead of da = (basic*15)/100;], then only T 2 will be rerun.
- So in the execution slice, we will have less number of statements.
- The execution slice is highlighted in the given code segment.

Begin

```

S1: read (basic, empid);
S2: gross=0;
S3: if(basic > 5000 || empid > 0)
    {
S4:   da = (basic*30)/100;
S5:   gross = basic + da;
    }
S6: else
    {
S7:   da = (basic*15)/100;
S8:   gross = basic + da;
    }
S9: print(gross, empid);

```

End

Dynamic Slice:

- The set of statements executed under a test case and have an effect on the program output under that test case is called **Dynamic Slice** of the program with respect to the output variables.

- For example, the following is the example program for Dynamic Slice

```

Begin
S1: read (a,b);
S2: sum=0;
S2.1: I=0;
S3: if (a==0)
{
    S4: print(b);
    S5: sum+=b;
}
S6: else if(b==0)
{
    S7: print(a);
    S8: sum+=a;
}
S9: else
{
    S10: sum=a+b+sum;
    S10.1 I=25;
    S10.2 print(I);
}
S11:endif
S12:print(sum);
End

```

Example program for dynamic slice

- The test cases for the given program.

Test Case	a	b	sum
T1	0	4	4
T2	67	0	67
T3	23	23	46

- T* 1, *T* 2, and *T* 3 will run correctly but if some modification is done in *S* 10.1 [say *I* = 50], then this change will not affect the output variable. Therefore, there is no need to rerun any of the test cases.
- On the other hand, if *S* 10 is changed [say, *sum*=*a***b*+*sum*], then this change will affect the output variable 'sum', so there is a need to rerun *T* 3.
- The dynamic slice is highlighted in the code segment.

```

Begin
S1: read (a,b);
S2: sum=0;
S2.1: I=0;
S3: if (a==0)
{

```

```

        S4: print(b);
        S5: sum+=b;
    }
    S6: else if(b==0)
    {
        S7: print(a);
        S8: sum+=a;
    }
    S9: else
    {
        S10: sum=a+b+sum;
        S10.1 I=25;
        S10.2 print(I);
    }
    S11:endif
S12:print(sum);
End

```

Relevant Slice:

- The set of statements that were executed under a test case and did not affect the output, but have potential to affect the output produced by a test case is known as *Relevant Slice* of the program.
- For example, in the above code, statements S2 , S4 have the potential to affect the output, if modified.

5.Prioritization based on Requirements

- This technique is used for prioritizing the system test cases.
- The system test cases are largely dependent on the requirements, the requirements can be analysed to prioritize the test cases.
- The approach for prioritizing the system test cases is known as PORT (prioritization of requirements for test).
- They have considered the following four factors for analysing and measuring the criticality of requirements:

Customer-assigned priority of requirements

- Based on priority, the customer assigns a weight (on a scale of 1 to 10) to each requirement.
- Higher the number, higher is the priority of the requirement.

Requirement volatility

- This is a rating based on the frequency of change of a requirement.
- The requirement with a higher change frequency is assigned a higher weight as compared to the stable requirements.

Developer-perceived implementation complexity

- All the requirements are not equal on a implementation level.
- The developer gives more weight to a requirement which he thinks is more difficult to implement

Fault proneness of requirements

- This factor is identified based on the previous versions of system.
- If a requirement in an earlier version of the system has more bugs, i.e. it is error-prone, then this requirement in the current version is given more weight.
- This factor cannot be considered for a new software.
- Based on these four factor values, a *prioritization factor value* (PFV) is computed as given below.

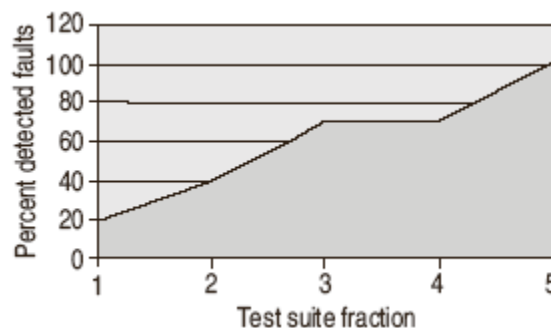
$$PFV_i = \sum(FV_{ij} \times FW_j)$$

where FV_{ij} = Factor value is the value of factor j corresponding to requirement i

FW_j = Factor weight is the weight given to factor j

6.Measuring the effectiveness of a Prioritized Test Suite

- When a prioritized test suite is prepared, the effectiveness of one prioritized test suite will we check we need one metric.
- For this purpose, the rate of fault-detection criterion can be taken. APFD (average percentage of faults detected) metric that measures the weighted average of the percentage of faults detected during the execution of a test suite.
- Its value ranges from 0 to 100, where a higher value means a faster fault-detection rate. APFD is a metric to detect how quickly a test suite identifies the faults.
- If we plot percentage of test suite run on the x-axis and percentage of faults detected on the y-axis, then the area under the curve is the APFD value shown below:



APFD is calculated as given below.

$$APFD = 1 - ((TF_1 + TF_2 + \dots + TF_m) / nm) + 1/2n$$

where: TF_i is the position of the first test in Test suite T that exposes fault i

m is the total number of faults exposed in the system or module under T

n is the total number of test cases in T

Example

Consider a program with 10 faults and a test suite of 10 test cases, as shown in the below Table

	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
F1					X			X		
F2		X	X	X		X				
F3	X	X	X	X			X	X		
F4						X				X
F5	X		X		X	X		X		X
F6					X	X	X		X	
F7									X	
F8		X		X		X			X	X
F9	X									X
F10			X	X					X	X

Test case fault exposure

If we consider the order of test suite as (T1, T2, T3, T4, T5, T6, T7, T8, T9, T10), then calculate the APFD for this program.

Solution

$$\begin{aligned} \text{APFD} &= 1 - ((5 + 2 + 1 + 6 + 1 + 5 + 9 + 2 + 1 + 3)/10 \times 10) + 1/2 \times 10 \\ &= 0.65 + 0.05 \\ &= 0.7 \end{aligned}$$

Software Quality Management

Software Quality Metrics

- Software Quality Metrics (standard of measurement) a subset of software metrics that focus on the quality aspects of the product, process and project.
- Software Quality Metrics can be divided into 3 groups:
 - Product Quality Metrics
 - In-Process Quality Metrics
 - Metrics for Software Maintenance

Product Quality Metrics

In product quality, the following metrics are considered

1. Mean-time to failure (MTTF)
2. Defect density metrics
3. Customer problem metrics
4. Customer satisfaction metrics

1.Mean time to failure (MTTF):

- **MTTF** metric is an estimate of the average or mean time until a product's first failure occurs. It is calculated by using:

$MTTF = \text{total operating time of the units tested} / \text{total number of failures encountered.}$

2.Defect Density Metrics:

- It measures the defects relative to the software size.

$\text{Defect Density} = \text{Number of Defects} / \text{Size of Product}$

- It is important for cost and resource estimate of the maintenance phase of the software lifecycle.
- It measures the defects relative to the software size expressed as lines of code or function point, etc. i.e., it measures code quality per unit. This metric is used in many commercial software systems.

3.Customer problem metrics:

- This metric measure the problems which customers face while using the product.
- The problems may be valid defects or usability problems, unclear documentation etc.
- The problems metric is usually expressed in terms of Problems per User Month (PUM).

$PUM = \text{Total problems reported by the customer for a time period} / \text{Total number of licensed months of the software during the period.}$

4.Customer Satisfaction metrics:

- Customer satisfaction is usually measured through various methods of customer surveys via the five point scale
- **Very Satisfied, Satisfied, Neutral, Dissatisfied, Very Dissatisfied.**

In-Process Quality Metrics

In-process quality metrics are less formally defined than the end-product metrics. Following are some metrics

1. **Defect Density during testing**
2. **Defect Arrival pattern during Testing**
3. **Defect Removal Efficiency**

1.Defect Density during testing:

- Higher defect rates found during testing is an indicator that the during its development process.
- This metric is a good indicator of quality while the software is still being tested.

2.Defect Arrival pattern during Testing:

- The pattern of defect arrivals or the time between consecutive failures gives more information.
- The objective is to have defect arrivals that give times between failures that are far apart.

3.Defect Removal Efficiency:

$DRE = (\text{Defects removed during a development phase} / \text{Defects latent in the product}) \times 100\%$

This metric can be calculated for the entire development process, for the front-end before code integration and for each phase.

It is called **early defect removal** when used for the front-end and **phase effectiveness** for specific phases.

3.Metrics for Software Maintenance

In this phase, the goal is to fix the defects as soon as possible and with excellent quality. The following metrics are important in software maintenance:

1.Fix backlog and backlog management index:

- Fix backlog is related to the rate of defect arrivals and the rate at which fixes for reported problems become available.
- Fix backlog metric is the count of reported problems that remain open at the end of a month or a week.
- Backlog management index (BMI) is also the metric to manage the backlog of open unresolved problems.

$BMI = (\text{Number of problems closed during the month} / \text{Number of problem arrivals during the month}) \times 100 \%$

- If BMI is larger than 100, it means the backlog is reduced; if BMI is less than 100, the backlog is increased. Therefore, the goal is to have a BMI larger than 100.

2.Fix response time and fix responsiveness:

- While fixing the problems, time-limit also matters according to the severity of the problems.
- The fix response time metric is used for this purpose and is calculated as mean-time of high severity defects from open to close. Thus, a short fix response time leads to customer satisfaction.
- This metric can be enhanced from the customers' viewpoint. Sometimes, the customer sets a time-limit to fix the defects.
- This agreed-to fix time is known as fix responsiveness metric.

3.Percent Delinquent fixes:

- For each fix, if the turn-around time greatly exceeds the required response time, then it is classified as delinquent.

Percent Delinquent fixes = (Number of fixes that exceeded the response time criteria by severity level / Number of fixes delivered in a specified time) X 100%

4.Fix Quality:

- It is the metric to measure the number of fixes that turn out to be defective.
- A fix is defective if it did not fix the reported problems a new defect as a consequence of fixing.
- Fix quality is measured as a percent defective fixes. Percent defective fixes is the percentage of all fixes in a time interval that are defective.

2.Software Quality Assurance (SQA) Models

- Organizations must adopt a model for standardizing their SQA activities.
- Many SQA models have been developed in recent years. Some of them are :

1.ISO 9126

2.Capability Maturity Model (CMM)

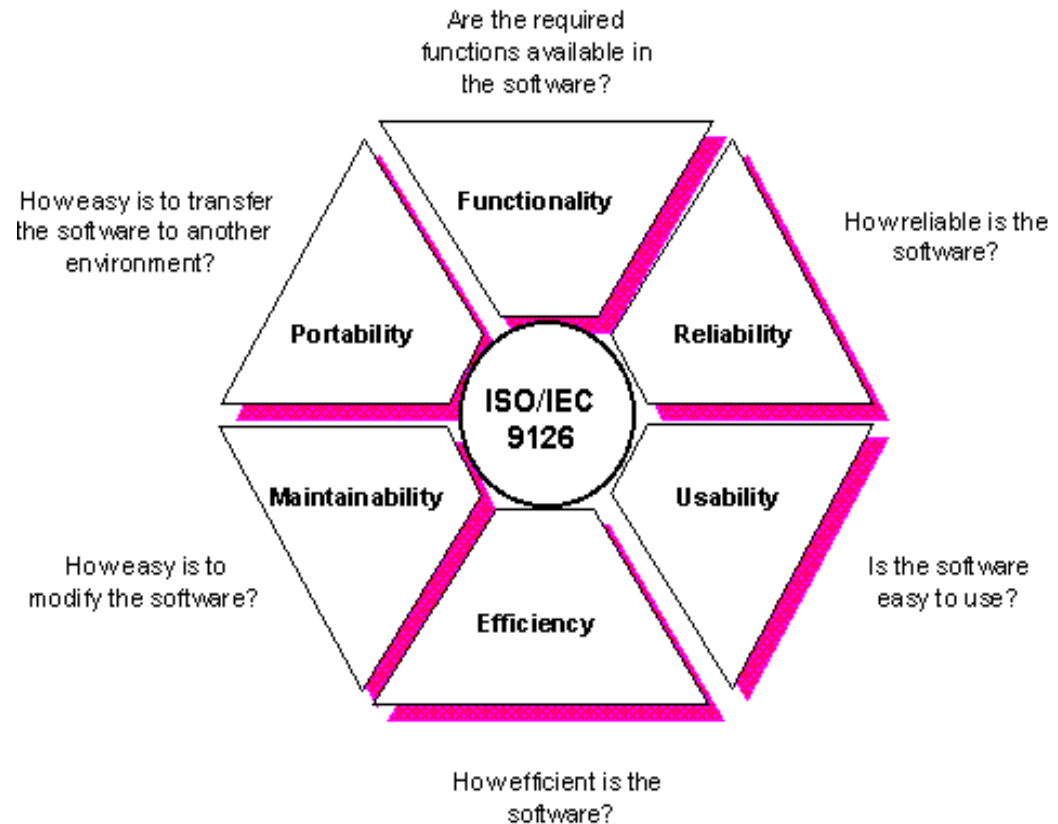
3.Software Total Quality Management (TQM)

4.Six Sigma

1.ISO 9126

- This is an international software quality standard from ISO. It aims at eliminating any misunderstanding between customers and developers.
- There may be a case that the customer is not able to communicate the requirements for the product to be developed. The developer should also be able to understand the requirement to provide the product with the right level of software quality.

- Thus, there may be problems with both customers and developers which may affect the quality of the software project.
- The ISO/IEC 9126 standard has been developed to address software quality issues.
- It specifies software product quality characteristics and sub-characteristics and proposes metrics for their evaluation.
- **ISO 9126 provides a set of six quality characteristics:** specifying the requirements, and evaluating the quality of software products throughout their life cycle.



ISO 9126 quality attributes

- ISO 9126 defines the following guidelines for implementation of this model:

Quality requirements definition

- A quality requirement specification is prepared with implied needs, technical documentation of the project, and the ISO standard.

Evaluation preparation

- The second stage is that of Evaluation preparation, which involves the selection of appropriate metrics, a rating level definition and the definition of assessment criteria.
- Metrics, in ISO 9126, give rise to measures mapped on to scales. The rating levels determine what ranges of values on those scales count as satisfactory or unsatisfactory.

- The assessment criteria definition involves preparing a procedure for summarizing the results of the evaluation.

Evaluation procedure

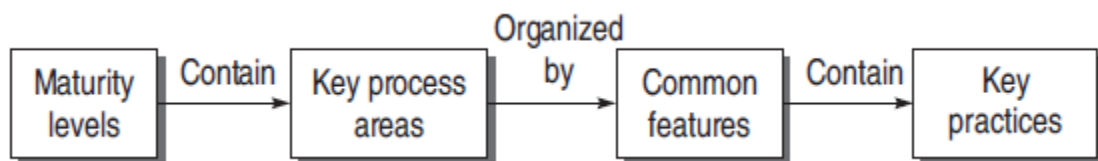
- The final stage is the evaluation procedure, which is refined into three steps: measurement, rating and assessment.
- In measurement, the selected metrics are applied to the software product and values on the scales of the metrics obtained.
- For each measured value, the rating level is determined.
- Assessment is the final step of the software evaluation process, where a set of rated levels are summarized.

2.Capability Maturity Model (CMM)

- Capability Maturity Model is a bench-mark for measuring the maturity of an organization's software process.
- CMM can be used to assess an organization against a scale of five process maturity levels based on certain Key Process Areas (KPA).
- It describes the maturity of the company based upon the project the company is dealing with and the clients.

CMM Structure

- The structure of CMM consists of five maturity levels.



CMM Structure

Maturity levels	Indicate	Process capability
Key process areas	Achieve	Goals
Common features	Address	Implementation/Institutionalization
Key practices	Describe	Infrastructure/Activities

Maturity Levels:

It is a layered framework providing a progression to the discipline needed to engage in continuous improvement.

Key Process Areas:

A Key Process Area (KPA) identifies a cluster of related activities that, when performed collectively, achieve a set of goals considered important.

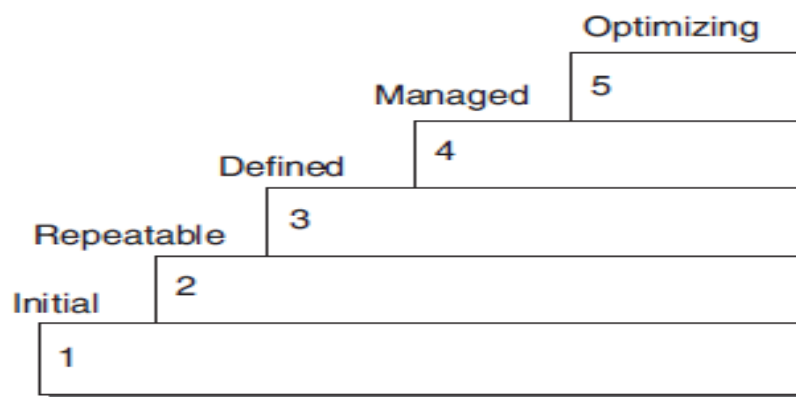
Common Features:

- Common features include practices that implement and institutionalize a key process area.
- These five types of common features include: Commitment to Perform, Ability to Perform, Activities Performed, Measurement and Analysis, and Verifying Implementation.

Key Practices:

The key practices describe the elements of infrastructure and practice that contribute most effectively to the implementation and institutionalization of the key process areas.

Maturity Levels



Maturity levels

Maturity level	Process Capability
1	—
2	Disciplined process
3	Standard consistent process
4	Predictable process
5	Continuously improving process

Initial:

- At this level, the process is ad-hoc, where there is no control on development progress and success is purely dependent on individual efforts.
- There is no stable environment for development. The team members may not know the components or goals of development processes.

Repeatable

- Earlier project successes are used here and the lessons learned from past projects are incorporated in the current project.

- Basic project management processes are established so that cost, schedule, and other parameters can be tracked.

Defined

- Here, standards for the processes of software development and maintenance are introduced and documented.

Managed

- A quantitative understanding of the processes is established in order to monitor and control them.
- Quantitative quality goals are also set for the processes, it means the processes are predictable and controlled.

Optimizing

- The process is continually improved through incremental and innovative technological changes or improvements.
- The main goal of an organization on this level is permanent improvement of existing processes.

Key Process Areas

The KPAs and their corresponding goals at each maturity level are follows:

<i>KPAs, at this level, focus on the project's concerns related to establishing basic project management controls.</i>	
KPA	Goals
Requirement Management	Document the requirements properly. Manage the requirement changes properly.
Software Project Planning	Ensure proper project plan including estimation and listing of activities to be done.
Software Project Tracking and Oversight	Evaluate the actual performance of the project against the plans during project execution. Action, if there is deviation from plan.
Software Sub-contract Management	Selects qualified software sub-contractors. Maintain ongoing communications between prime contractor and the software sub-contractor. Track the software sub-contractor's actual results and performance against its commitments.
Software Quality Assurance	Plan the SQA activities. Ensure that there are proper processes by conducting review and audits. Take proper actions, if projects fail.
Software Configuration Management	Identify work products and documents to be controlled in the project. Control the changes in the software.

KPAs for level 2

KPAs, at this level, address both project and organizational issues, as the organization establishes an infrastructure that institutionalizes effective software engineering and management processes across all projects.

KPA	Goals
Organization Process Focus	Coordinate process development and improvement activities across the organization. Compare software processes with a process standard. Plan organization-level process development and improvement activities.
Organization Process Definition	Standard software processes are defined and documented.
Training Program	Identify training needs of various team members and implementation of training programs.
Integrated Software Management	Tailor the project from the standard defined process. Manage the project according to defined process.
Software Product Engineering	Define, integrate, and perform software engineering tasks. Keep the work products consistent especially if there are changes.
Inter-group Coordination	Ensure coordination between different groups.
Peer Reviews	Plan peer review activities. Identify and remove defects in the software work products.

KPAs for level 3

KPAs, at this level, focus on establishing a quantitative understanding of both the software process and the software work products being built.

KPA	Goals
Quantitative Process Management	Plan the quantitative process management activities. Control the performance of the project's defined software process quantitatively. The process capability of the organization's standard software process is known in quantitative terms.
Software Quality Management	Set and plan quantitative quality goals for the project. Measure the actual performance of the project with quality goals and compare them with plans.

KPAs for level 4

KPAs, at this level, cover the issues that both the organization and the projects must address to implement continuous and measurable software process improvement.

KPA	Goals
Defect Prevention	Plan the defect prevention activities. Identify, prioritize, and eliminate the common causes of bugs.
Technology Change Management	Plan the technology changes to be incorporated in the project, if any. Evaluate the effect of new technologies on quality and productivity.
Process Change Management	Plan the process improvement activities such that an organization-wide participation is there. Measure the change of improvement in the process.

KPAs for level 5

Common Features

- All KPAs are organized by common features. The common features are attributes that indicate what to implement according to a KPA goal
- The common features have been listed below:

Common Feature	What to Implement/Institutionalize
Commitment to Perform	Organization must take action to ensure that the process is established and will carry on.
Ability to Perform	Define the pre-conditions that must exist in the project or organization to implement the software process competently. Estimate resources, organizational structures, and training.
Activities Performed	Assign the team members and identify procedures to implement a key process area. Perform the work, track it, and take corrective actions as necessary.
Measurement and Analysis	Measure the process and analyse the results.
Verifying Implementation	Ensure that the activities are performed in compliance with the process that has been established. Implement reviews and audits by management and software quality assurance team.

List of common features

Assessment of Process Maturity

- The assessment is done by an assessment team consisting of some experienced team members.

- These members start the process of assessment by collecting information about the process being assessed.
- The information about the process can be collected in the form of maturity questionnaires, documentation, or interviews.

3. Software Total Quality Management (TQM)

- Total quality management is a widely used philosophy and business approach that requires all departments in an organization to participate in continuous quality improvement efforts.
- A TQM values high performance and avoids, or at least minimizes, waste.
- Most companies, especially manufacturers in regulated environments, use TQM software (or TQM system) to help them instill total quality management procedures in all aspects of their operations.
- TQM that exists in the market today is designed based on total quality management principles that can be found in quality standards and regulations.
- TQM is defined as a quality-centered, customer-focused, fact-based, team-driven, senior management led process to achieve an organization's strategic through continuous process improvement.

T = Total = **everyone** in the organization

Q = Quality = **customer satisfaction**

M = Management = **people** and **processes**

- The elements of a TQM system can be summarized as follows:

Customer focus

- The goal is to attain total customer satisfaction.
- It includes studying customers' needs, gathering customer requirements, and measuring and managing customer satisfaction.

Process

- The goal is to achieve continuous process improvement.
- Process includes both the business process and the product development process.

Human-side of quality

- The goal is to create a company-wide quality culture.
- Focus areas include management commitment, total participation, employee empowerment, and other social, psychological, and human factors.

Measurement and analysis

- The goal is to drive continuous improvement in all quality parameters by the goal-oriented measurement system.

4.Six Sigma

- Six Sigma is a methodology for pursuing continuous improvement in customer satisfaction and profit.
- Six Sigma is a highly disciplined process that helps us focus on developing and delivering near-perfect products and services.

Features of Six Sigma

- Six Sigma's aim is to eliminate waste and inefficiency, thereby increasing customer satisfaction by delivering what the customer is expecting.
- Six sigma derives its meaning from the field of statistics. Sigma is the standard deviation for a statistical population.
- Six sigma means, if one has six standard deviations between the mean of a process and the nearest specification limit, there will be practically no items that fail to meet the specifications.
- Today, you can apply six sigma to many fields like services, medical, and insurance procedures, call centers including software.
- Six sigma processes will produce less than 3.4 defects per million opportunities. To achieve this target, it uses a methodology known as DMAIC with the following step
 - Define opportunities
 - Measure performance
 - Analyse opportunity
 - Improve performance
 - Controlperformance

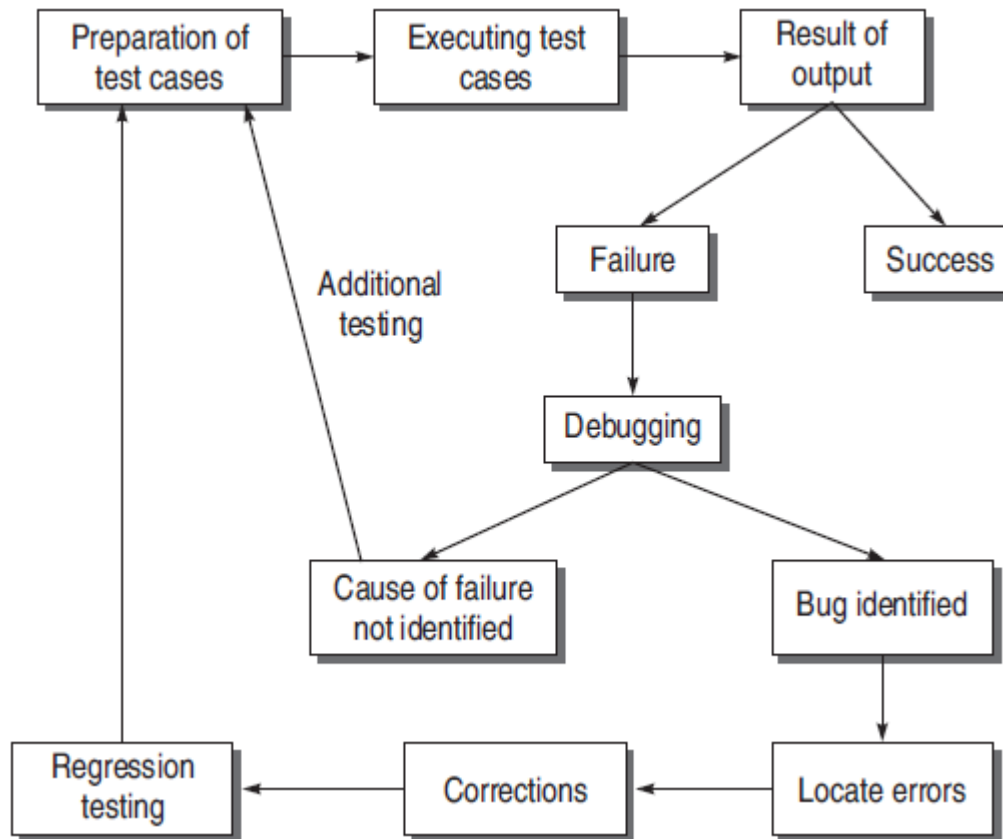
. Debugging

- Debugging is not a part of the testing domain.
- It is a separate process performed as a consequence of testing. But the testing process is considered a waste if debugging is not performed after the testing.
- If output of testing is not successful, then a failure has occurred. Now the goal is to find the bugs that caused the failure and remove the errors present in the software. This is called **debugging**.
- Debugging is the process of identification of the symptoms of failures, tracing the bug, locating the errors that caused the bug, and correcting these errors.

Process

- The goal of debugging process is to determine the exact nature of failure with the help of symptoms identified, locate the bugs and errors, and finally correct it.

- The debugging process is explained in the following steps:



Debugging process

- Check the result of the output produced by executing test cases prepared in the testing process. If the actual output matches with the expected output, it means that the results are successful. Otherwise, there is failure in the output which needs to be analysed.
- Debugging is performed for the analysis of failure that has occurred, where we identify the cause of the problem and correct it. It may be possible that symptoms associated with the present failure are not sufficient to find the bug. Some additional testing is required so that we may get more clues to analyse the causes of failures.
- If symptoms are sufficient to provide clues about the bug, then the cause of failure is identified. The bug is traced to find the actual location of the error.
- Once we find the actual location of the error, the bug is removed with corrections.
- Regression testing is performed as bug has been removed with corrections in the software.

Debugging Techniques

1. Debugging with Memory Dump

- In this technique a printout of all registers and relevant memory locations is obtained and studied.

- The relevant data of the program is observed through these memory locations and registers for any bug in the program.
- Following are the drawbacks of this method:
 - There is difficulty of establishing the correspondence between storage locations and the variable in one's source program.
 - It shows the state of the program at only one instant of time.
 - The Storage locations are often represented in octal or hexadecimal formats.

2.Debugging with Watch Points

- At a particular point of execution in the program, value of variables or other actions can be verified.
- These particular points of execution are known as **watch points**.
- Debugging with watch points can be implemented with the following methods:

Output Statements

- In this method, output statements can be used to check the state of a condition or a variable at some watch point in the program.
- Therefore, output statements are inserted at various watch points; program is compiled and executed with these output statements.
- Execution of output statements may give some clues to find the bug.
- This method has the following drawbacks:
 - It may require many changes in the code. These changes may mask an error or introduce new errors in the program.
 - After analysing the bug, we may forget to remove these added statements which may cause other failures or misinterpretations in the result.

Breakpoint Execution

- This is the advanced form of the watch point used with an automated debugger program.
- Breakpoint is actually a watch point inserted at various places in the program.
- Breakpoints allow the programmer to control execution of the program and specify how and where the application will stop to allow further examination. Breakpoints have an obvious advantage over output statements.
- Removing the breakpoints are easy when compared to output statements.
- There is no need to compile the program after inserting breakpoint, while it is necessary for output statement.

Breakpoints can be categorized as:

Unconditional Breakpoint: It is a simple breakpoint without any condition to be evaluated.

Conditional Breakpoint:

- On the activation of this breakpoint, one expression is evaluated for its boolean value.

Temporary Breakpoint:

- This breakpoint is used only once in the program. When it is set, the program starts running, and once it stops, it is removed.

Single Stepping

- The idea of single stepping is that the users should be able to watch the execution of the program after every executable instruction.
- After every instruction execution, the users can watch the status of variable
- Single stepping is implemented with the help of internal breakpoints.

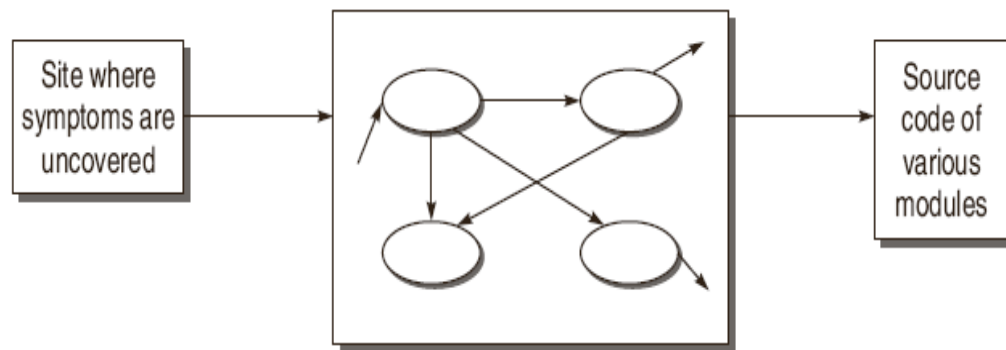
Step-into

- It means execution proceeds into any function in the current source statement and stops at the first executable source line in that function.

Step-over

- It is also called skip, instead of step.
- It treats a call to a function as an atomic operation and proceeds past any function calls to the textually succeeding source line in the current scope.

3.Back Tracking



Backtracking

This is a logical approach for debugging a program. The following are the steps for backtracking process

1. Observe the symptom of the failure at the output side and reach the site where the symptom can be uncovered. For example, suppose you are not getting display of a value on the output. After examining the symptom of this problem, you uncover that one module is not sending the proper message to another module. This can be uncovered using the above mentioned techniques, e.g. using breakpoints.

2. Once you have reached the site where symptom has been uncovered, trace the source code starting backwards and move to the highest level of design. For tracing backwards, collect source code of all the modules which are related at this point.
3. Slowly isolate the module through logical backtracking using data flow diagrams (DFDs) of the modules wherein the bug resides.
4. Logical backtracking in the isolated module will lead to the actual bug and error can thus be removed.

Correcting Bugs

- The second phase of the debugging process is to correct the error when it has been uncovered.
- The design of the software should not be affected by correcting the bug or due to new modifications.
- Before correcting the errors, we should concentrate on the following points:
 - Highly coupled module correction can introduce many other bugs. That is why low-coupled module is easy to debug.
 - After recognizing the influence of corrections on the other modules, plan the regression tests.
 - Perform regression testing with every correction in the software to ensure that the corrections have not introduced new bugs.

Debugging Guidelines

Fresh thinking leads to good debugging

- Don't spend continuous hours and hours on debugging a problem. The continuous involvement in the problem will collapse your efficiency.
- So rest in between and start with a fresh mind to concentrate on the problem.

Don't isolate the bug from your colleagues

- It has been observed that bugs cannot be solved in isolation. Show the problem to other colleagues, explain to them what is happening, and discuss the solution, there will be a lot of clues.
- Therefore, it is always better to share the problem to others.

Don't attempt code modifications in the first attempt

- Debugging always starts with the analysis of clues.
- If you don't analyse the failures and clues and simply change the code randomly with the view it will lead you with junk code added to the software.

Additional test cases are a must if you don't get the symptom or clues to solve the problem

- Design test cases with the view to execute those parts of the program which are causing the problem.
- This test case execution will provide symptoms to be analysed.

Regression testing is a must after debugging

- The process of debugging does not end with fixing the error. After fixing the error, you need to realize the effects of this change on other parts.
- Test cases must be designed to uncover any effects on bugs produced on other parts.
- That is why regression testing is necessary after fixing the errors.

Design should be referred before fixing the error

- Any change in the code to fix the bug should be according to pre-specified design of the software.
- While making corrections, that design should not be violated by any means.